Kolmogorov–Arnold Networks and Applications to Derivative Pricing

Vladislav Terentev

ICEF

28.05.2026

Overview

B-Spline Basis Functions

- Start with a knot vector:

$$\xi_0 < \xi_1 < \cdots < \xi_m.$$

- The order-zero B-spline is just an indicator of one knot interval:

$$B_{i,0}(x) = \begin{cases} 1, & \xi_i \le x < \xi_{i+1}, \\ 0, & \text{otherwise.} \end{cases}$$

- Higher-order B-splines are defined recursively:

$$B_{i,k}(x) = \frac{x - \xi_i}{\xi_{i+k} - \xi_i} B_{i,k-1}(x) + \frac{\xi_{i+k+1} - x}{\xi_{i+k+1} - \xi_{i+1}} B_{i+1,k-1}(x).$$

- Thus $B_{i,k}$ is a local degree-k polynomial basis function.

Spline as a Combination of B-Splines

- Once the knot grid and order k are fixed, the basis functions

$$B_{1,k}(x),\ B_{2,k}(x),\ \ldots,\ B_{M,k}(x)$$

  are fixed.

- A spline is a linear combination of these basis functions:

$$s(x) = \sum_{i=1}^{M} c_i B_{i,k}(x).$$

- The trainable objects are the coefficients:

$$c_1, c_2, \ldots, c_M$$

- Changing one coefficient c_i only affects a local part of the curve, because $B_{i,k}$ is nonzero only near a few neighboring knots.

Small Example: Quadratic Spline, $k = 2$

Suppose we observe points

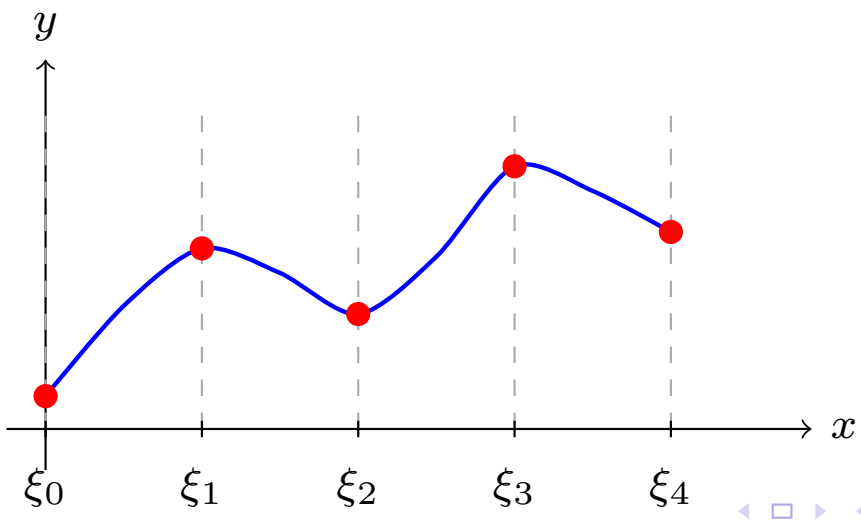
x_j	0	1	2	3	4
y_j	0.2	1.1	0.7	1.6	1.2

and choose knots

$$\xi_0 = 0, \quad \xi_1 = 1, \quad \xi_2 = 2, \quad \xi_3 = 3, \quad \xi_4 = 4.$$

A quadratic spline has the form

$$s(x) = \sum_i c_i B_{i,2}(x), \qquad B_{i,2}(x) = 0 \quad \text{outside} \quad [\xi_i, \xi_{i+3}].$$



MLP: General One-Hidden-Layer Case

- Input: $x \in \mathbb{R}^n$.
- Hidden units $i = 1, \ldots, H$:

$$h_i(x) = \sigma(w_i \cdot x + b_i).$$

- Output:

$$f(x) = \sum_{i=1}^{H} a_i h_i(x) = \sum_{i=1}^{H} a_i \sigma(w_i \cdot x + b_i).$$

- **Fixed:** the activation function σ (e.g. ReLU, $\tanh$).
- **Learned:** output coefficients a_i, directions w_i, and biases b_i.

Universal Approximation Theorem (UAT)

Theorem

For a continuous target function $f : K \subset \mathbb{R}^n \to \mathbb{R}$ and any tolerance $\varepsilon > 0$, there exists a one-hidden-layer MLP such that

$$f(x) \approx \sum_{i=1}^{N(\varepsilon)} a_i \, \sigma(w_i \cdot x + b_i),$$

with approximation error below ε on K.

- $N(\varepsilon)$: required width depends on the desired accuracy.
- The theorem is **existential**.

What KANs Help MLPs Overcome

The UAT shows that an MLP can approximate f, but it does not control how $N(\varepsilon)$ scales with dimension.

Two practical weaknesses of MLPs:

- **Curse of dimensionality:** generic fully connected approximation may require rapidly growing width in high dimension.
- **Fixed activations:** ReLU or $\tanh$ units can be inefficient for accurate smooth or oscillatory univariate functions.

KANs attack both points:

- compositional structure is learned through sums of univariate edge functions,
- each edge function is a spline, so local 1D approximation is accurate and adjustable.

Kolmogorov–Arnold Representation Theorem

Theorem

For a continuous target function $f : [0,1]^n \to \mathbb{R}$, there exist continuous univariate functions

$$\phi_{q,p} : [0,1] \to \mathbb{R}, \qquad \Phi_q : \mathbb{R} \to \mathbb{R},$$

such that

$$f(x) = f(x_1, \ldots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right).$$

- In the classical theorem learned functions may be highly irregular.
- The theorem is also **existential**

Supervised Learning Task

- Suppose we have input-output pairs $\{(x_i, y_i)\}_{i=1}^{N}$.
- We want to find a function f such that

$$y_i \approx f(x_i)$$

  for all data points.
- The Kolmogorov–Arnold representation above implies that we are done if we can find suitable univariate functions $\phi_{q,p}$ and Φ_q.

KAN Layer: Edge Functions

- A KAN architecture is described by layer widths

$$[n_0, n_1, \ldots, n_L],$$

  where n_ℓ is the number of nodes in layer ℓ.

- Let $x_{\ell,i}$ be the activation value of node i in layer ℓ:

$$x_\ell = (x_{\ell,1}, \ldots, x_{\ell,n_\ell})^\top.$$

- Between layer ℓ and layer $\ell + 1$, every connection has its own trainable univariate function:

$$\phi_{\ell,j,i}, \qquad i = 1, \ldots, n_\ell, \quad j = 1, \ldots, n_{\ell+1}.$$

- The input to this edge function is just the source-node activation: $x_{\ell,i}$.
- The output of the edge function is $\widetilde{x}_{\ell,j,i} = \phi_{\ell,j,i}(x_{\ell,i})$.

KAN Layer: Summation and Matrix Form

- Node j in the next layer sums all incoming edge outputs:

$$x_{\ell+1,j} = \sum_{i=1}^{n_\ell} \widetilde{x}_{\ell,j,i} = \sum_{i=1}^{n_\ell} \phi_{\ell,j,i}(x_{\ell,i}), \qquad j = 1, \dots, n_{\ell+1}.$$

- This can be written in matrix-like form:

$$x_{\ell+1} = \underbrace{\begin{pmatrix} \phi_{\ell,1,1}(\cdot) & \phi_{\ell,1,2}(\cdot) & \cdots & \phi_{\ell,1,n_\ell}(\cdot) \\ \phi_{\ell,2,1}(\cdot) & \phi_{\ell,2,2}(\cdot) & \cdots & \phi_{\ell,2,n_\ell}(\cdot) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{\ell,n_{\ell+1},1}(\cdot) & \phi_{\ell,n_{\ell+1},2}(\cdot) & \cdots & \phi_{\ell,n_{\ell+1},n_\ell}(\cdot) \end{pmatrix}}_{\Phi_\ell} x_\ell.$$

Thus, the matrix entries are trainable one-dimensional functions.

From MLPs to KANs

MLP

$$\mathrm{MLP}(x) = (\mathbf{W}_{L-1} \circ \sigma \circ \mathbf{W}_{L-2} \circ \sigma \circ \cdots \circ \mathbf{W}_1 \circ \sigma \circ \mathbf{W}_0)x.$$

$$x_{\ell+1} = \sigma(\mathbf{W}_\ell x_\ell).$$

- Learnable objects: scalar weights in $\mathbf{W}_\ell$.
- Nonlinearity: fixed activation σ on nodes.

KAN

$$\mathrm{KAN}(x) = (\mathbf{\Phi}_{L-1} \circ \mathbf{\Phi}_{L-2} \circ \cdots \circ \mathbf{\Phi}_1 \circ \mathbf{\Phi}_0)x.$$

$$x_{\ell+1,j} = \sum_{i=1}^{n_\ell} \phi_{\ell,j,i}(x_{\ell,i}), \qquad j = 1, \ldots, n_{\ell+1}.$$

- Learnable objects: univariate edge functions $\phi_{\ell,j,i}$.
- Node operation: summation of incoming edge activations.

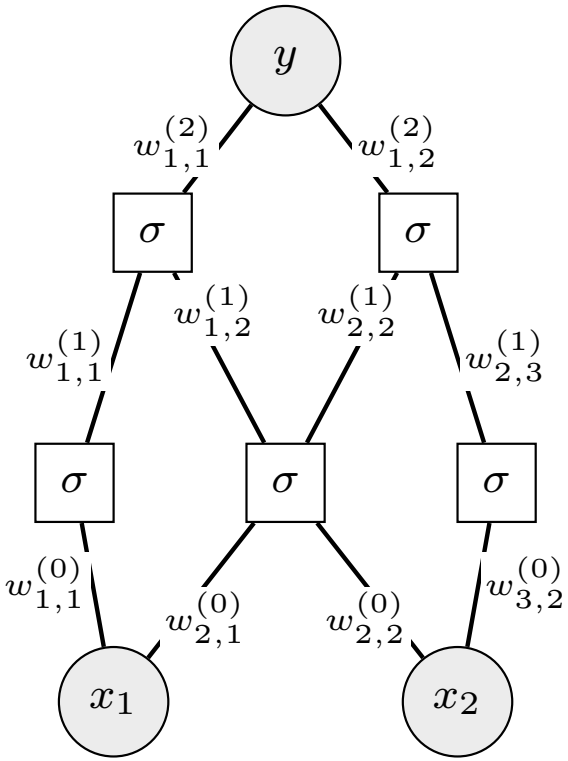
MLP vs KAN: Algorithmic Cost and Modeling Difficulty

Aspect	MLP	KAN
Parameter / memory scale	For depth L, width N: $O(N^2 L)$	For spline grid G, order k: $O(N^2 L(G+k)) \approx O(N^2 LG)$
Raw compute per same width	Cheaper: scalar weights and fixed activations	More expensive: each edge stores/evaluates a spline
Approximation difficulty	May need large width/depth for smooth or oscillatory functions	Learns univariate functions directly through splines
Scaling issue	UAT guarantees existence, but not favorable $N(\varepsilon)$ scaling	Can scale better when a smooth compositional representation exists
Interpretability	Weights are hard to interpret directly	Edge functions can be plotted as learned 1D transformations

Key trade-off: KANs are heavier per edge, but may need a much smaller computation graph than MLPs.

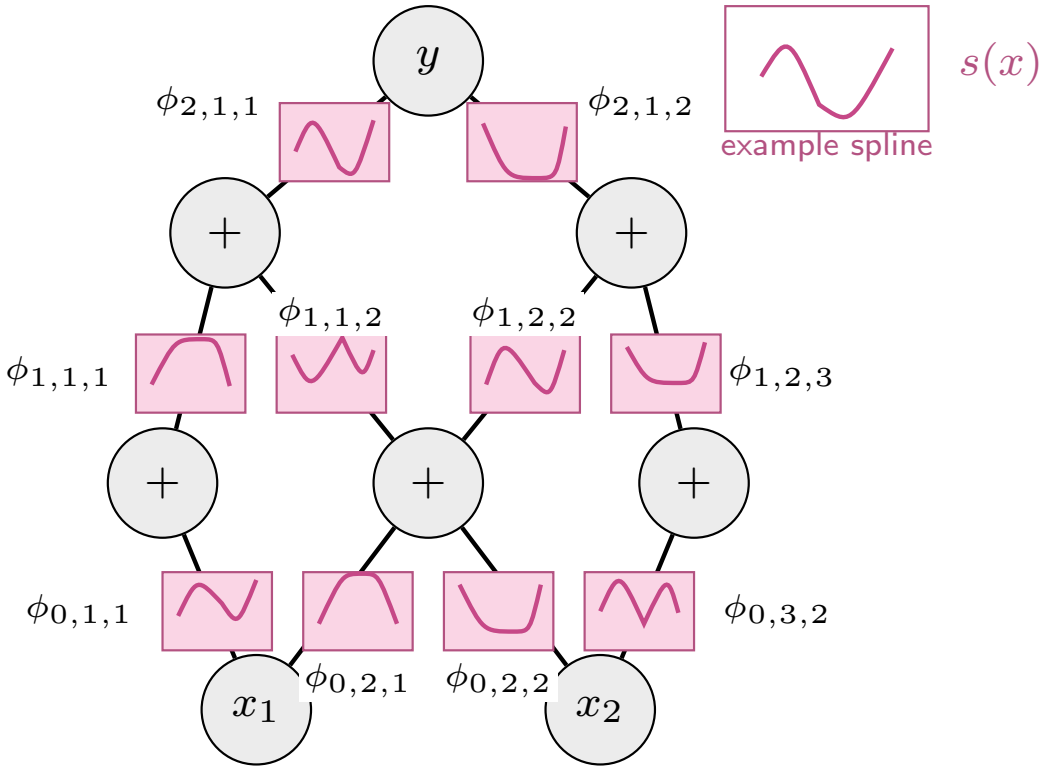
MLP vs KAN: Fixed vs Trainable Activations

MLP



fixed node activation σ
train scalar weights $w^{(\ell)}_{j,i}$

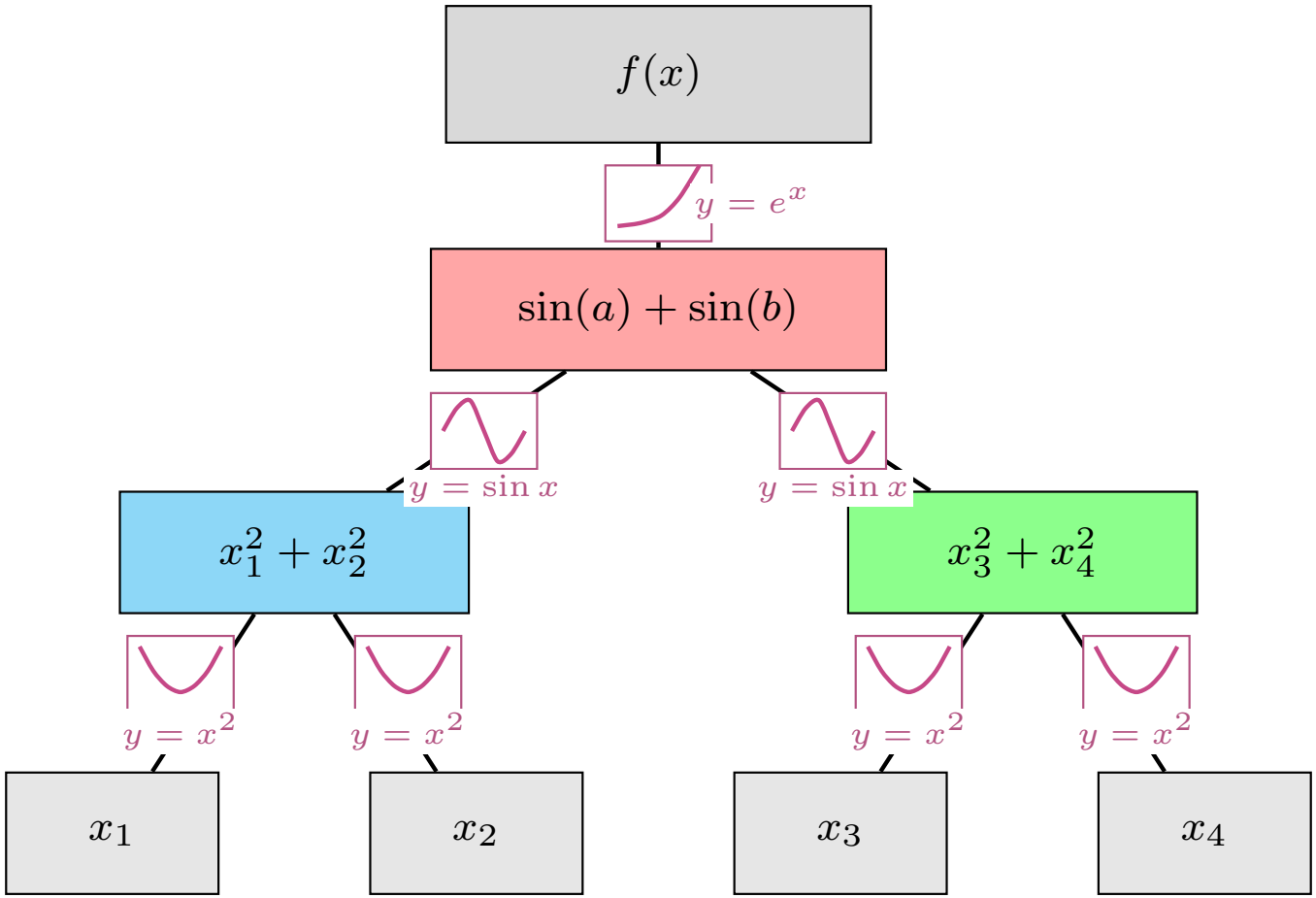
KAN



nodes only sum
train edge functions $\phi_{\ell,j,i}$

Example: KAN Representing a Compositional Function

$$f(x_1, x_2, x_3, x_4) = \exp\big(\sin(x_1^2 + x_2^2) + \sin(x_3^2 + x_4^2)\big)\,.$$



This is a $[4, 2, 1, 1]$ KAN

KAN PDE Example: Poisson Equation

Liu et al. solve a controlled Poisson boundary-value problem on

$$\Omega = [-1, 1]^2, \qquad z = (x, y).$$

The PDE is

$$\begin{aligned} u_{xx}(x, y) + u_{yy}(x, y) &= f(x, y), \quad (x, y) \in \Omega, \\ u(x, y) &= 0, \qquad\quad (x, y) \in \partial\Omega. \end{aligned}$$

They choose the exact solution first:

$$u(x, y) = \sin(\pi x) \sin(\pi y^2).$$

Then the right-hand side is manufactured as

$$f(x, y) = u_{xx}(x, y) + u_{yy}(x, y).$$

KAN PDE Example: Verifying the Exact Solution

For

$$u(x, y) = \sin(\pi x) \sin(\pi y^2),$$

the second derivatives are

$$u_{xx} = -\pi^2 \sin(\pi x) \sin(\pi y^2),$$

and

$$u_{yy} = 2\pi \sin(\pi x) \cos(\pi y^2) - 4\pi^2 y^2 \sin(\pi x) \sin(\pi y^2).$$

Therefore,

$$\boxed{f(x, y) = -\pi^2(1 + 4y^2) \sin(\pi x) \sin(\pi y^2) + 2\pi \sin(\pi x) \cos(\pi y^2)}$$

The boundary condition also holds because

$$\sin(\pi x) = 0 \text{ at } x = \pm 1, \qquad \sin(\pi y^2) = 0 \text{ at } y = \pm 1.$$

Thus $u = 0$ on $\partial\Omega$.

KAN PDE Example: Physics-Informed Loss

- The KAN represents the unknown PDE solution: $u_\theta(x, y) \approx u(x, y)$
- At interior points $z_i = (x_i, y_i) \in \Omega$, the PDE residual is

$$R_\theta(z_i) = \frac{\partial^2 u_\theta}{\partial x^2}(z_i) + \frac{\partial^2 u_\theta}{\partial y^2}(z_i) - f(z_i),$$

  where f is fixed by the PDE.
- The interior loss forces the learned function to satisfy the PDE:

$$L_{\text{int}}(\theta) = \frac{1}{n_i} \sum_{i=1}^{n_i} |R_\theta(z_i)|^2.$$

- At boundary points $z_j^b \in \partial\Omega$, the boundary loss enforces $u = 0$:

$$L_{\text{bdry}}(\theta) = \frac{1}{n_b} \sum_{j=1}^{n_b} |u_\theta(z_j^b)|^2.$$

- The KAN parameters are chosen by minimizing

$$\theta^* = \arg\min_\theta L_{\text{pde}}(\theta), \qquad L_{\text{pde}}(\theta) = \alpha L_{\text{int}}(\theta) + L_{\text{bdry}}(\theta).$$

Notation for Finance-Informed Pricing

Symbol		Meaning
S		asset price
$\tau = T - t$		time to maturity used in implementation
$C(S, \tau)$		European call price, unknown target function
$v_\theta(S, \tau)$		KAFIN normalized approximation of $C(S, \tau)/K$
$C_\theta(S, \tau)$ $K v_\theta(S, \tau)$	$=$	option price implied by the network output
$F[\cdot; \lambda]$		generic financial differential operator with model parameters λ
K, r, σ, T		strike, interest rate, volatility, maturity
$S_{\max}$		upper bound of the computational asset-price domain
$R_\theta(S, \tau)$		Black–Scholes PDE residual computed from C_θ
θ		trainable neural-network parameters

KAFIN: Finance-Informed KAN

- KAFIN approximates an unknown financial quantity by a KAN:

$$C_\theta(x, \tau) \approx C(x, \tau),$$

  where $C(x, \tau)$ can be an option price or another financial variable.

- Instead of fitting only observed labels, KAFIN trains C_θ to satisfy financial laws:

$$F[C(x, \tau); \lambda] = 0, \qquad x \in \Omega, \quad \tau \in [0, T].$$

- The generic objective is

$$\theta^* = \arg\min_\theta \left[\lambda_{\text{initial}} L_{\text{initial}} + \lambda_{\text{boundary}} L_{\text{boundary}} + \lambda_{\text{financial}} L_{\text{financial}}\right].$$

- Here $L_{\text{financial}}$ penalizes the residual of the governing financial operator F, while the other terms enforce initial and boundary conditions.

KAFIN Problem: European Call Pricing

- KAFIN outputs a normalized price v_θ, and the implied option price is

$$C_\theta(S,\tau) = K v_\theta(S,\tau) \approx C(S,\tau),$$

  where S is the asset price and $\tau = T - t$ is time to maturity.

- In time-to-maturity form, the Black–Scholes PDE is

$$C_\tau - \frac{1}{2}\sigma^2 S^2 C_{SS} - rSC_S + rC = 0.$$

- Here:

$$C_\tau = \frac{\partial C}{\partial \tau}, \qquad C_S = \frac{\partial C}{\partial S}, \qquad C_{SS} = \frac{\partial^2 C}{\partial S^2}.$$

- The unknown function is $C(S,\tau)$. KAFIN replaces it with the trainable price surface $C_\theta(S,\tau) = K v_\theta(S,\tau)$.

KAFIN Training: Black–Scholes Residual

- Substitute the implied price $C_\theta(S,\tau) = K v_\theta(S,\tau)$ into the Black–Scholes operator:

$$R_\theta(S,\tau) = \frac{\partial C_\theta}{\partial \tau} - \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 C_\theta}{\partial S^2} - rS\frac{\partial C_\theta}{\partial S} + rC_\theta.$$

- If C_θ exactly solved the PDE, then

$$R_\theta(S,\tau) = 0$$

  at all interior points.

- Therefore, the finance-informed loss penalizes PDE violations:

$$L_{\text{financial}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} R_\theta(S_i,\tau_i)^2.$$

- Training adjusts θ so that the learned price surface $C_\theta(S,\tau)$ approximately satisfies the Black–Scholes PDE.

Payoff, Boundary Conditions, and Total Loss

- The payoff anchors the solution at maturity, where $\tau = 0$:

$$C(S, 0) = \max(S - K, 0).$$

- The corresponding payoff loss is

$$L_{\text{payoff}} = \frac{1}{M} \sum_{i=1}^{M} \big(C_\theta(S_i, 0) - \max(S_i - K, 0)\big)^2.$$

- Boundary conditions for a European call:

$$C(0, \tau) = 0, \qquad \lim_{S \to \infty} C(S, \tau) = S - Ke^{-r\tau}.$$

- On the finite computational domain $S \in [0, S_{\max}]$, use

$$C(S_{\max}, \tau) \approx S_{\max} - Ke^{-r\tau}.$$

- The total finance-informed objective is

$$\theta^* = \arg\min_\theta \left[\lambda_{\text{financial}} L_{\text{financial}} + \lambda_{\text{payoff}} L_{\text{payoff}} + \lambda_{\text{boundary}} L_{\text{boundary}}\right].$$

- The benchmark is the analytical BS price evaluated on a fixed grid.

Appendix: Weierstrass Models and Results

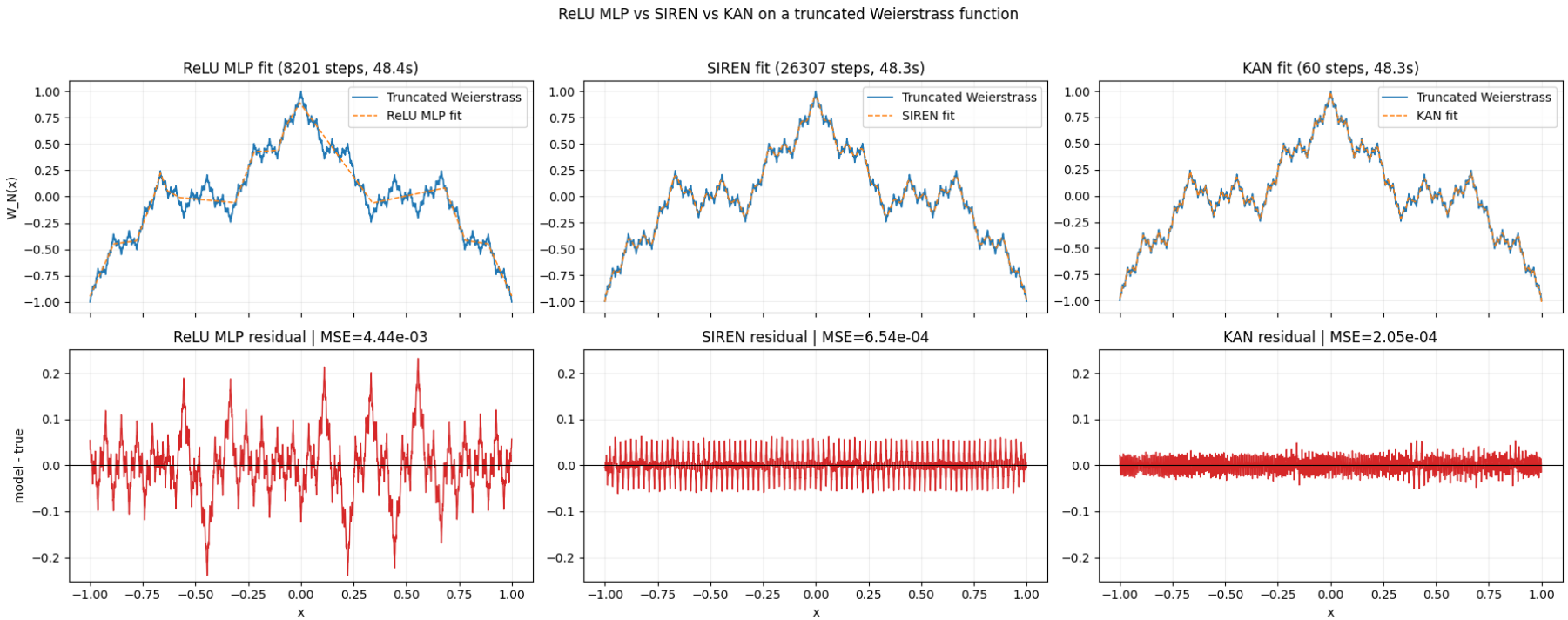
Target:

$$W_N(x) = \frac{1}{\sum_{n=0}^{N-1} a^n} \sum_{n=0}^{N-1} a^n \cos(b^n \pi x), \qquad a = 0.5,\ b = 3,\ N = 7.$$

Model	Architecture	Mathematical form	Test MSE
ReLU MLP	$1 \to 32 \to 16 \to 1$	$h_{\ell+1} = \mathrm{ReLU}(W_\ell h_\ell + b_\ell)$	4.443×10^{-3}
SIREN	$1 \to 32 \to 16 \to 1$ $\omega_0 = 20$	$h_{\ell+1} = \sin(\omega_0 W_\ell h_\ell + b_\ell)$	6.541×10^{-4}
KAN	$1 \to 32 \to 16 \to 1$ grid 21, cubic $k = 3$	$\phi(x) = w_b b(x) + w_s \sum_i c_i B_i(x)$	2.050×10^{-4}

Training: 8000 train points, 2000 test points; equal wall-clock budget ≈ 48.3s.

Example: KAN on a Truncated Weierstrass Function



KAN achieves the smallest residual with far fewer optimization steps than the MLP and SIREN baselines and same training time*

Appendix: Black–Scholes Models and Results

All models output a normalized price v_θ; the option price used in the Black–Scholes loss is

$$C_\theta(S,\tau) = K\, v_\theta(\tilde{S},\tilde{\tau}).$$

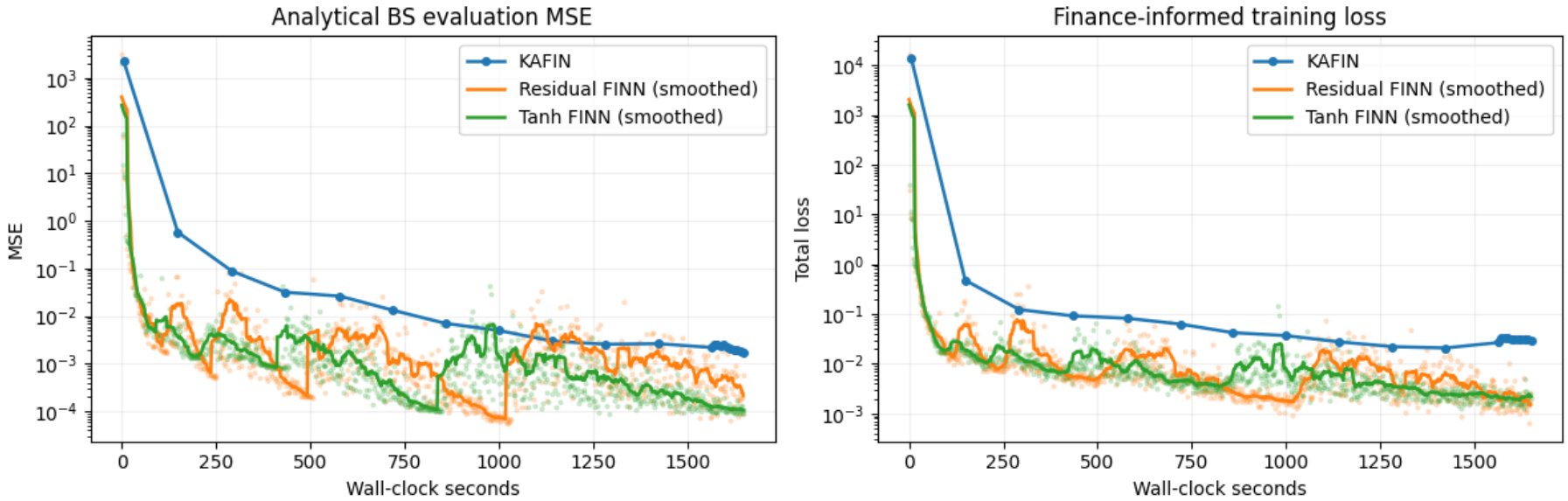
PDE residual and loss, with τ as time to maturity:

$$R_\theta = (C_\theta)_\tau - \frac{1}{2}\sigma^2 S^2 (C_\theta)_{SS} - rS(C_\theta)_S + rC_\theta, \qquad \mathcal{L} = 0.1\,\mathbb{E}[R_\theta^2] + \mathcal{L}_{\text{payoff}} + \mathcal{L}_{S=0} + \mathcal{L}_{S=S_{\max}}.$$

Model	Architecture	Mathematical form	BS-grid error
Tanh FINN	$2 \to 32 \to 32 \to 1$	$h_{\ell+1} = \tanh(W_\ell h_\ell + b_\ell)$	MSE 9.060×10^{-5} MAE 5.961×10^{-3}
Residual FINN	$2 \to 32 \to 32 \to 1$ two residual blocks	$h_{\ell+1} = h_\ell + \tanh(W_\ell h_\ell + b_\ell)$	MSE 5.736×10^{-4} MAE 1.936×10^{-2}
KAFIN	$2 \to 32 \to 16 \to 1$ cubic splines, 24 bases	$v_\theta = (\Phi_2 \circ \Phi_1 \circ \Phi_0)(\tilde{S},\tilde{\tau})$, $C_\theta = K v_\theta$	MSE 1.759×10^{-3} MAE 3.099×10^{-2}

$K = 60$, $r = 0.05$, $\sigma = 0.20$, $T = 1$, $S \in [0, 160]$; equal wall-clock budget ≈ 1649s

Black–Scholes Training Dynamics



Under the same wall-clock budget, KAFIN takes far fewer but heavier optimization steps, while cheaper FINN baselines can continue improving through many more updates*.